// 2026.03.30

[PROOF OF CONCEPT]

AUTOMATED FORENSIC ANALYSIS FRAMEWORK

DIGITAL FORENSICS TOOLKIT

ENGINEER

File Carving // Memory Analysis // Steganography // Metadata

BASH BINWALK VOLATILITY FOREMOST EXIFTOOL STEGSEEK

Denis Albin

albindeniswork@gmail.com

Cybersecurity Student | Digital Forensics

01 // Executive Summary

Engineer is an automated forensic analysis tool written in Bash, designed to streamline the investigation of hard drive images, memory dumps, and suspicious files. It provides a unified interface for 10+ forensic tools, from file carving and metadata extraction to steganography detection and memory analysis with Volatility 3. All operations are logged with timestamps for audit compliance and chain of custody.

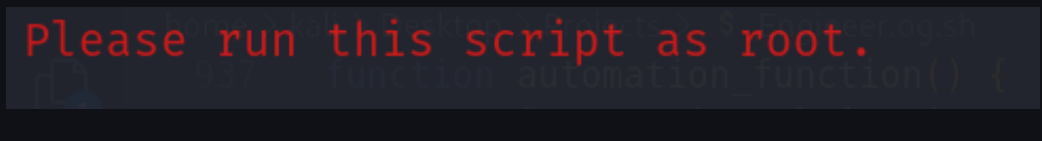


Fig 1. Root privilege enforcement — script refuses to run without sudo

KEY CAPABILITIES

- Root permission verification before forensic operations
- Automated installation of 10 forensic tools (binwalk, exiftool, volatility3, etc.)
- File carving with Foremost and Scalpel for deleted file recovery
- Metadata extraction via Exiftool with save-to-file option
- Steganography detection using Stegseek against wordlists
- Memory analysis with Volatility 3 (Windows and Linux OS detection)
- Bulk Extractor for email, credit card, URL, AES key, and PCAP recovery
- Full automation mode: runs all tools in sequence with zipped report output

TOOL STACK

Binwalk | Exiftool | Foremost | Scalpel | Strings | DD | Stegseek | Volatility3 | Bulk Extractor | Hexdump

WORKFLOW

Root Check → Tool Install → File Select → Analyze → Carve/Extract → Report

02 // Root Check & File Selection

The script begins with **root_check()**, verifying administrative privileges required for forensic tools like Volatility and raw disk access. If not running as root, the script displays a red warning and exits immediately — ensuring all subsequent forensic operations have the permissions they need. The **file_check()** function then provides an interactive file browser with directory navigation, letting the user locate and select the target file for analysis. It validates file existence and displays its path before proceeding to the main menu.

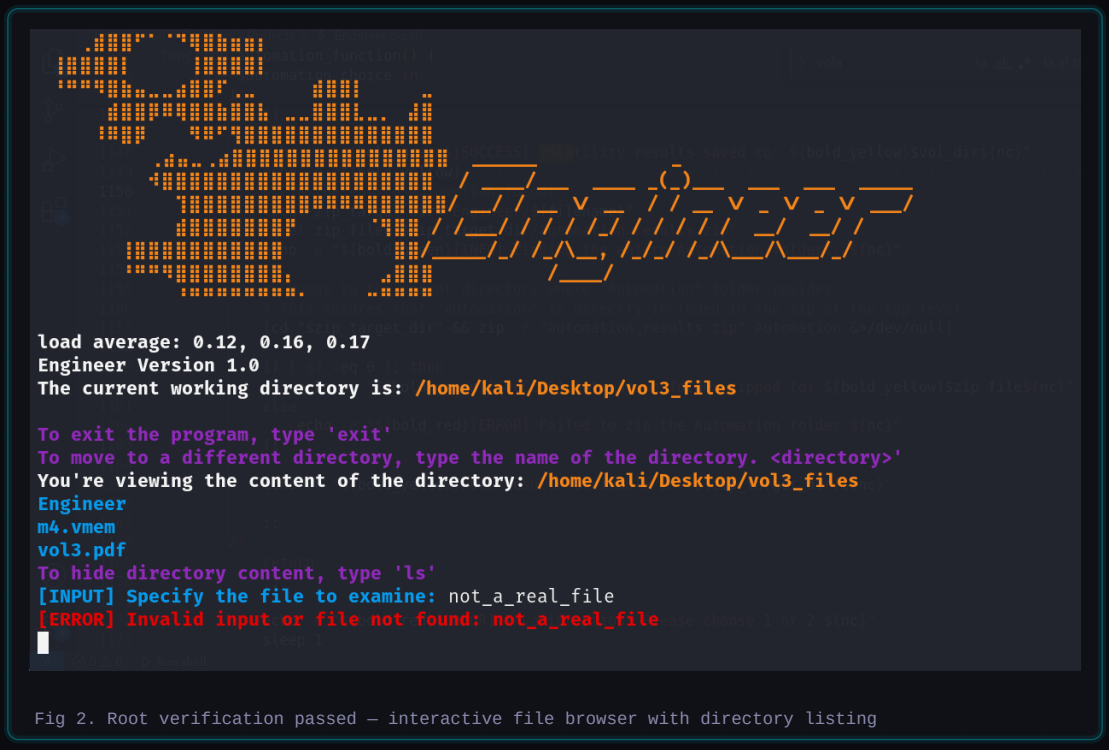


Fig 2. Root verification passed — interactive file browser with directory listing

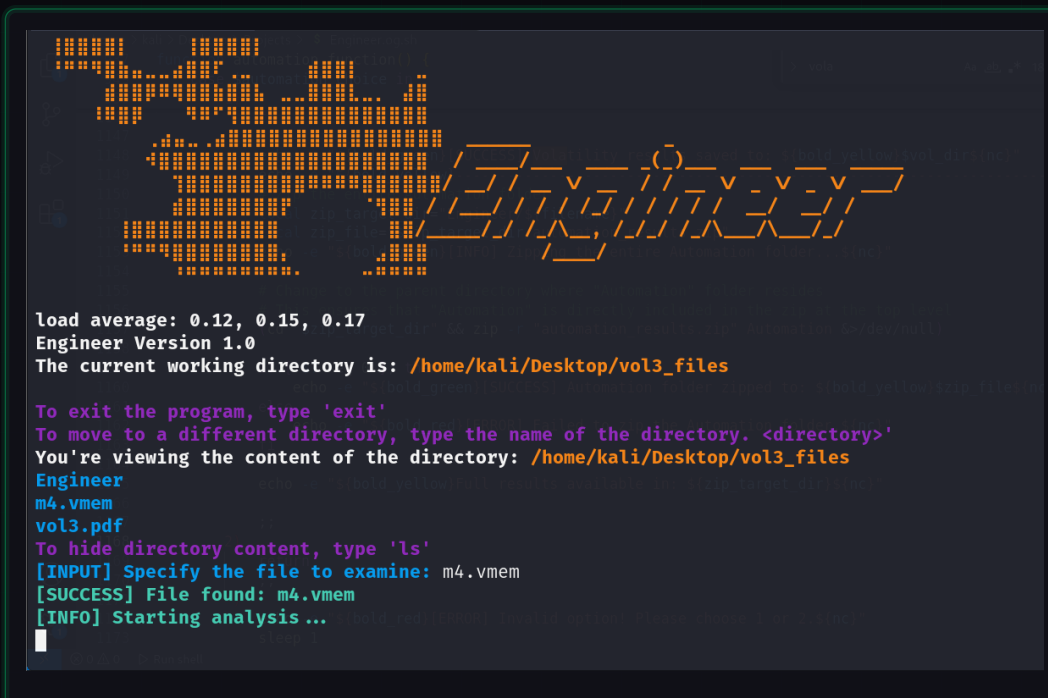


Fig 3. Target file successfully located and selected for forensic analysis

03 // Tool Installation & Forensic Mode

The **tool_check()** function iterates over all required forensic tools and verifies their installation via **command -v**. Any tool not found is automatically installed through **apt-get install**. Volatility 3 receives special handling — it's installed via pip3 with the **--break-system-packages** flag to avoid virtualenv issues on Kali Linux. The installation check runs before every session, ensuring the full toolkit is always available regardless of the system's starting state.

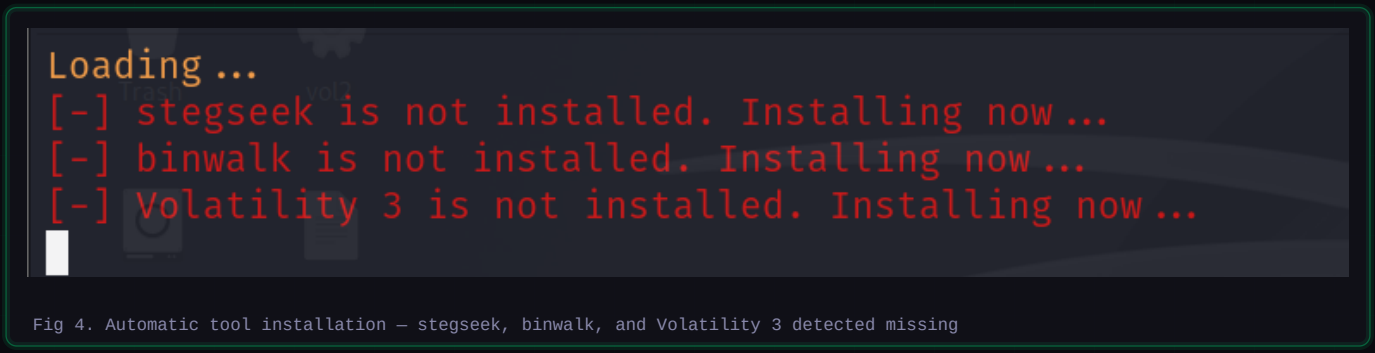


Fig 4. Automatic tool installation — stegseek, binwalk, and Volatility 3 detected missing

FORENSIC ANALYSIS MODE

Once tools are verified and a target file is selected, the script enters Forensic Analysis Mode. A status banner displays the selected file's metadata — filename, size (via exiftool), last modification date, and detected file type — providing an at-a-glance summary before analysis begins. This banner appears throughout the analysis session for quick reference.



Fig 5. Forensic Analysis Mode banner — file metadata at a glance

MAIN MENU ARCHITECTURE

The main menu presents **13 analysis options** organized by forensic discipline: Binwalk analysis (quick scan, extract, entropy), Exiftool metadata extraction, Foremost and Scalpel file carving, Strings with keyword filtering, DD byte-level extraction, Hexdump canonical display, Stegseek steganography detection, Volatility 3 memory analysis, Bulk Extractor comprehensive artifact recovery, and a full automation mode that chains every tool together. Each option has its own sub-menu with save (-s) capabilities.

DIRECTORY STRUCTURE

```
Engineer/<filename>/<tool>/  — each tool writes to its own subdirectory
Timer tracks analysis duration per tool, all results timestamped for audit trail
```

04 // Strings Analysis

The **strings_function()** extracts human-readable text from binary files, providing a powerful interface for keyword-based filtering. Users enter search terms interactively — common forensic keywords include 'password', 'username', 'login', 'key', 'secret', and 'admin'. The function pipes the target file through **strings** and **grep -a** to perform pattern matching across binary-safe output. Results can be saved to Engineer/<filename>/Strings/ with the -s flag. Users can run multiple searches in a loop, typing 'exit' to return to the main menu.

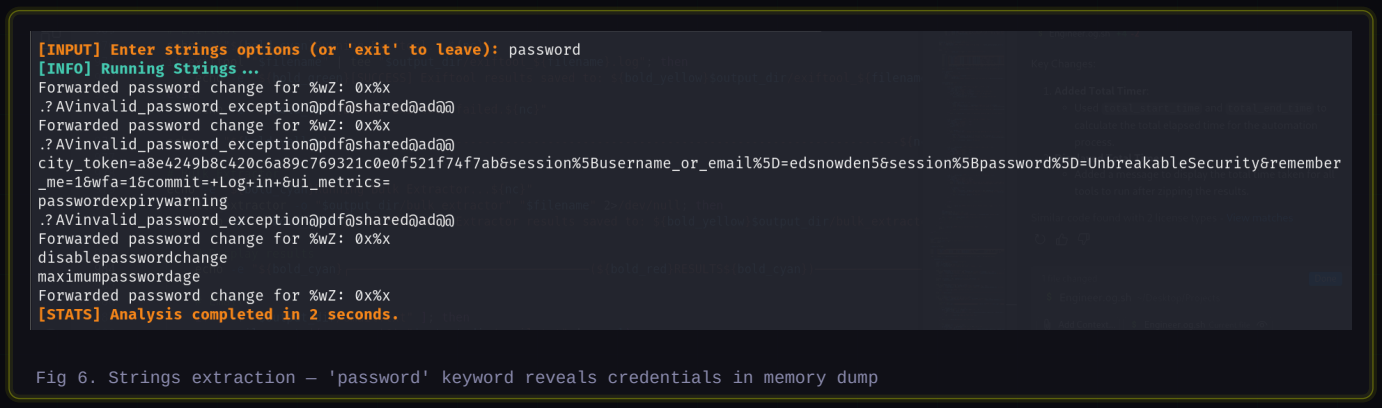


Fig 6. Strings extraction — 'password' keyword reveals credentials in memory dump

FORENSIC VALUE

String extraction is often the first step in forensic triage. Memory dumps and disk images frequently contain plaintext credentials, session tokens, URLs, and configuration data. In the example above, the strings search revealed multiple forwarded password changes, session tokens with Base64-encoded credentials, and authentication cookies — all critical indicators of compromise that would warrant deeper investigation with specialized tools.

STEGANOGRAPHY DETECTION

The **stegseek_function()** performs high-speed dictionary attacks against steghide-protected image files. Stegseek can test millions of passwords per second, making it highly effective for discovering hidden content embedded within JPEG files. When a passphrase is cracked, the hidden data is extracted and saved alongside the detection log. This module is essential for detecting covert data exfiltration through image steganography.

DD EXTRACTION & HEXDUMP

dd_function() — precise byte-level extraction using decimal offsets from binwalk

hexdump -C — canonical hex+ASCII display for manual byte-level inspection

Both tools output to structured directories with save options for audit trails

05 // Memory Analysis — OS Detection

The **vol_function()** provides a complete Volatility 3 integration for memory dump analysis. It starts with automatic OS detection by first attempting **windows.info** — if that returns results, the dump is classified as Windows and the Windows plugin set becomes available. If windows.info fails, the function falls back to **linux.pslist** to test for Linux memory structures. This two-stage detection is critical because Windows and Linux memory layouts are fundamentally different, and using the wrong plugin set will return garbage data or errors.



Fig 7. Successful OS detection — Windows memory dump identified via windows.info

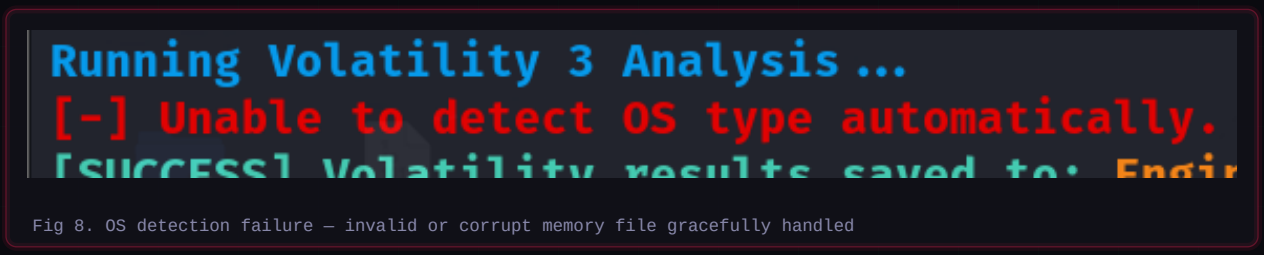


Fig 8. OS detection failure — invalid or corrupt memory file gracefully handled

VOLATILITY 3 SUB-MENU

After OS detection, the Volatility sub-menu provides four analysis options tailored to the detected operating system. Each option is mapped to OS-specific plugins:

OS-SPECIFIC PLUGIN MAPPING

WINDOWS:	windows.info | windows.pslist | windows.netscan | windows.registry.printkey
LINUX:	linux.info (+ kallsyms fallback) | linux.pslist | linux.sockstat

NOTES:

- Registry extraction is Windows-only (via printkey). Linux suggests bash/kmsg instead
- Network: netscan (Windows) vs sockstat (Linux). Automation adds malfind + check_tty
- All results can be saved with -s flag to Engineer/<filename>/Volatility/

06 // Process Analysis

The process listing module uses **windows.pslist** or **linux.pslist** depending on the detected OS. It extracts every running process from the memory dump at the time of capture, displaying critical fields: PID (Process ID), PPID (Parent PID), ImageFileName, number of threads, number of handles, session ID, and whether the process is flagged as Wow64 (32-bit process on 64-bit system). This data is essential for identifying suspicious processes — malware often masquerades as legitimate system processes with slightly misspelled names (e.g., 'svchost' vs 'svch0st'), runs from unusual parent processes, or spawns an abnormal number of child threads.

```
  [0×01] | 1 Find Memory Profile      | Detect memory profile
  [0×02] | 2 List Processes           | Display running processes
  [0×03] | 3 Network Connections      | Display network connections
  [0×04] | 4 Extract Registry         | Extract registry information
  [0×05] | 5 Back to Volatility Main  | Return to Volatility OS check menu

Note: Add '-s' after the option number to save results (e.g., '1 -s').
[INPUT] Select an option (1-5) and optionally add '-s': 2
[INFO] Listing running processes ...
Volatility 3 Framework 2.11.0

PID     PPID    ImageFileName   Offset(V)       Threads Handles SessionId       Wow64   CreateTime      ExitTime        File output

4       0       System  0×808ccda83480  119     -       N/A     False   2024-01-29 11:35:14.000000 UTC  N/A     Disabled
284     4       smss.exe        0×808ccee2f040  2       -       N/A     False   2024-01-29 11:35:14.000000 UTC  N/A     Disabled
400     384     csrss.exe       0×808cced5f5c0  10      -       0       False   2024-01-29 11:35:15.000000 UTC  N/A     Disabled
480     284     smss.exe        0×808ccf7605c0  0       -       1       False   2024-01-29 11:35:16.000000 UTC  2024-01-29 11:35:16.000000 UTC  D
isabled
488     384     wininit.exe     0×808ccf7635c0  1       -       0       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
496     480     csrss.exe       0×808ccf7725c0  12      -       1       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
580     480     winlogon.exe    0×808ccf7965c0  5       -       1       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
624     488     services.exe    0×808ccf7e55c0  8       -       0       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
644     488     lsass.exe       0×808ccf7f8340  9       -       0       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
736     624     svchost.exe     0×808ccf847080  25      -       0       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
748     488     fontdrvhost.ex  0×808ccf84f5c0  5       -       0       False   2024-01-29 11:35:16.000000 UTC  N/A     Disabled
```

Fig 9. Complete process listing from Windows memory dump — PIDs, handles, and thread counts

THREAT HUNTING INDICATORS

- Unusual parent-child relationships (e.g., explorer.exe spawning cmd.exe → powershell.exe)
- Processes running from temp directories or user-writable locations
- Abnormally high handle/thread counts may indicate process injection
- System processes (csrss, lsass, svchost) should only have specific known parents

07 // Network Connection Analysis

Network analysis uses **windows.netscan** (Windows) or **linux.sockstat** (Linux) to reconstruct the active network state at the time of memory capture. For Windows dumps, netscan reveals all TCP and UDP connections including local/remote addresses, ports, connection state (ESTABLISHED, LISTENING, CLOSE_WAIT, TIME_WAIT), the owning process PID, and creation timestamp. This is one of the most critical forensic artifacts — it reveals active C2 (Command & Control) channels, data exfiltration connections, lateral movement attempts, and any listening backdoors the attacker may have established.



Fig 10. Network connections — TCP/UDP state, remote IPs, ports, and owning processes

KEY NETWORK INDICATORS OF COMPROMISE

- ESTABLISHED connections to unknown external IPs — potential C2 communication
- LISTENING ports on unusual numbers — possible backdoor or reverse shell
- High-volume connections from a single process — data exfiltration pattern

08 // Registry Extraction

For Windows memory dumps, the registry extraction module uses **windows.registry.printkey** to recover registry hives directly from volatile memory. The Windows Registry is a hierarchical database storing system configuration, installed software, user preferences, network settings, and — critically for forensics — evidence of malicious persistence mechanisms. Attackers commonly use registry Run/RunOnce keys for persistence, modify service configurations, or store encoded payloads in custom registry values. This module dumps the full registry tree, enabling the analyst to search for autorun entries, recently accessed files, USB device history, and more.

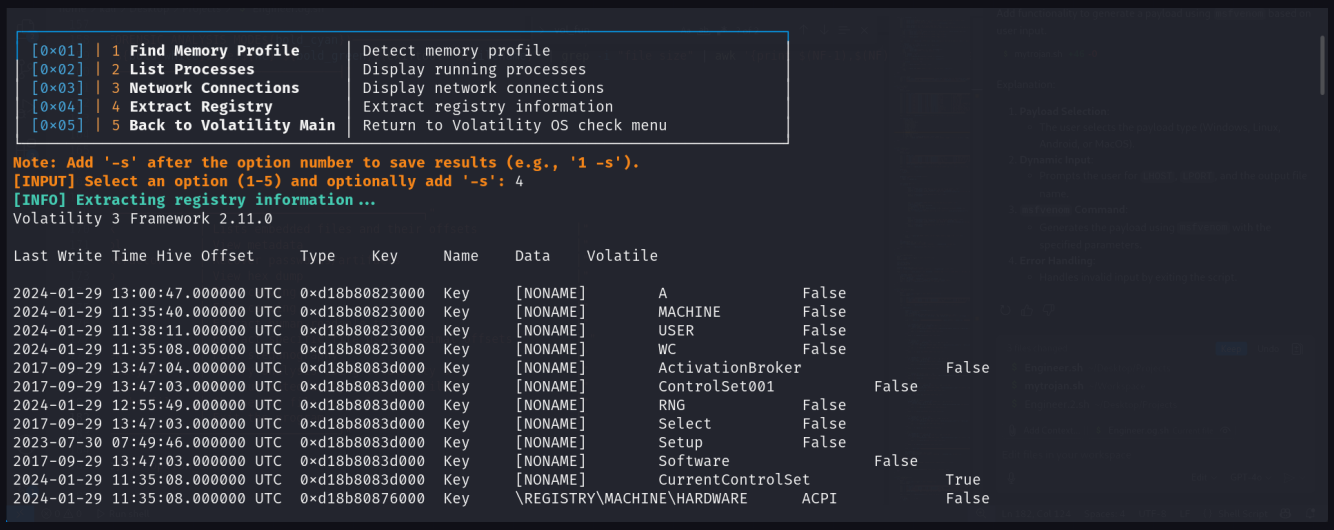


Fig 11. Windows registry extraction — hive keys, values, and data types from memory

FORENSIC REGISTRY ARTIFACTS

- HKLM\Software\Microsoft\Windows\CurrentVersion\Run — auto-start programs (persistence)
- HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs — user file access
- HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR — USB device connection history
- HKLM\SYSTEM\CurrentControlSet\Services — installed services (malware persistence)

09 // Bulk Artifact Extraction

The **bulk_extractor_menu()** leverages the Bulk Extractor tool for comprehensive, high-speed artifact recovery from disk images and memory dumps. Unlike traditional carving tools that look for file headers, Bulk Extractor scans raw bytes for structured data patterns — email addresses, credit card numbers, URLs, PCAP network packets, EXIF metadata, Base64-encoded data, AES encryption keys, and JSON structures. It processes data in parallel using multiple threads, making it exceptionally fast even on large evidence files. Each artifact type is checked individually, with match counts reported in real time.

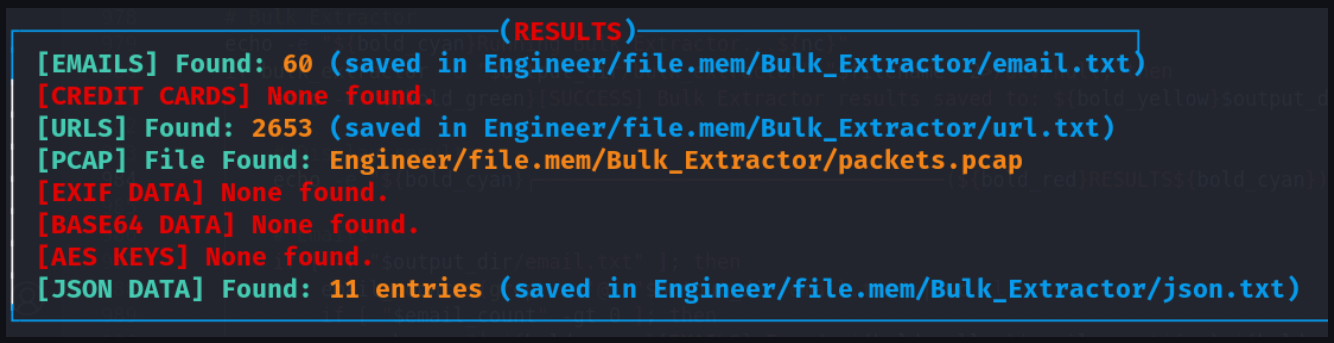


Fig 12. Bulk Extractor results — emails, URLs, and PCAP packets recovered from evidence

ARTIFACT TYPES SCANNED

Emails | Credit Cards | URLs | PCAP | EXIF | Base64 | AES Keys | JSON

Each artifact type is checked and counted individually. Zero-match types are reported as 'not found' rather than silently skipped — ensuring nothing is overlooked.

PCAP recovery enables network traffic reconstruction from memory dumps — invaluable for incident response

REAL-WORLD APPLICATIONS

In incident response scenarios, Bulk Extractor is often the single most valuable tool in the arsenal. A memory dump from a compromised workstation can yield the attacker's email addresses, C2 server URLs, exfiltrated credit card data, and even reconstructed network sessions via PCAP fragments. The Engineer script wraps this powerful tool with an intuitive menu, automatic output organization, and summary reporting — making it accessible even for junior analysts.

10 // Full Automation & Reporting

The **automation_function()** is Engineer's most powerful feature — a single command that executes the entire forensic toolkit in sequence against the target file. It runs: Binwalk (signature scan), Exiftool (full metadata dump), Bulk Extractor (all artifact types with detailed counting), Foremost (file carving), Scalpel (signature-based recovery), Strings (with a predefined set of forensic keywords like password, token, shell, admin, etc.), Stegseek (steganography detection), and Volatility 3 (with automatic OS detection and all applicable plugins including malfind for Windows). A master timer tracks the total analysis duration across all tools.

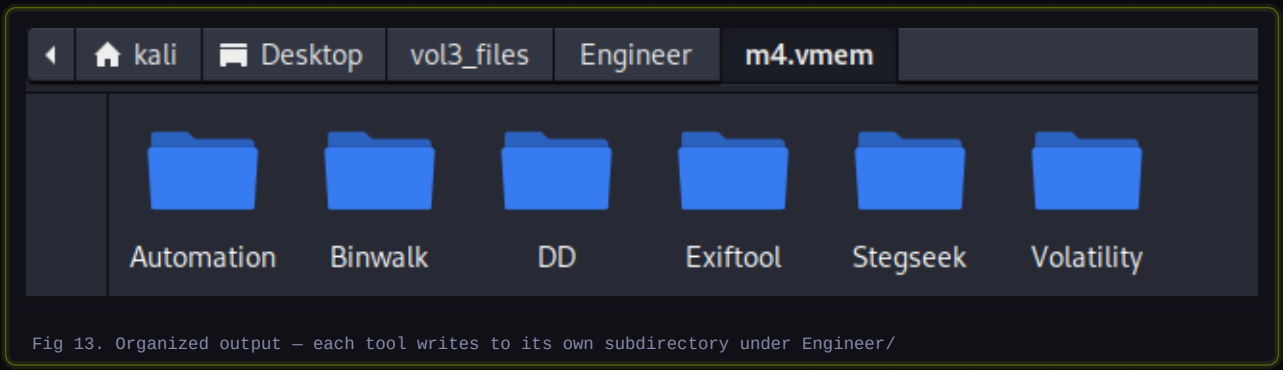


Fig 13. Organized output — each tool writes to its own subdirectory under Engineer/

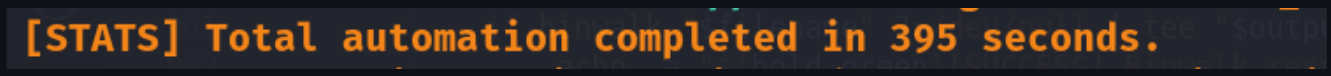


Fig 14. Automation complete — full analysis finished in 395 seconds

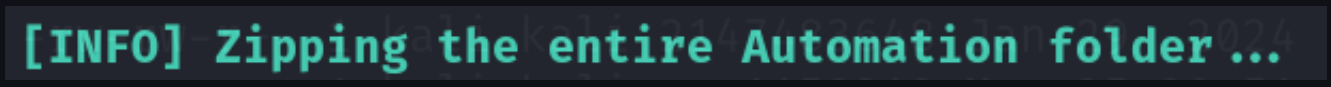
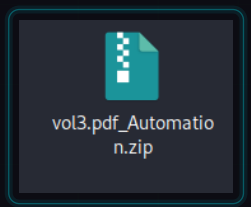


Fig 15. Results automatically compressed into a portable ZIP archive



PORTABLE EVIDENCE PACKAGE

Fig 16. The final ZIP archive — ready for sharing and archival

All results compressed into <basename>_Automation.zip
Contains: binwalk logs, exiftool data, carved files, strings, volatility reports, bulk extractor artifacts, and malfind results
Suitable for chain of custody documentation and evidence transfer

11 // Technical Deep Dive

FILE CARVING ENGINE

Foremost and Scalpel perform header/footer-based file carving to recover deleted files. Foremost uses built-in signatures while Scalpel allows custom signature definitions via its configuration file. Both output recovered files into organized directory structures. The script fixes a common Scalpel pitfall — using **-i** for input and **-o** for output (not the reversed flags found in many tutorials).

BINWALK ANALYSIS

Three modes: Quick Scan (**-B**) for signature detection, Extract (**-e**) for automated file extraction with --run-as=root, and Entropy Graph (**-E**) for compression/encryption detection. High entropy regions suggest encrypted or compressed data within the target file. The entropy visualization is invaluable for identifying packed malware or encrypted partitions.

METADATA EXTRACTION

Exiftool extracts comprehensive metadata: file type, size, modification dates, camera info, GPS coordinates, software versions, and document properties. This metadata can reveal authorship, editing history, device information, and geographic data — often the first pivot point in an investigation that links a file to a specific person, device, or location.

MEMORY FORENSICS

Volatility 3 integration handles both Windows and Linux memory dumps with automatic OS detection. Windows analysis: windows.info, pslist, netscan, registry.printkey, and malfind (malware detection). Linux analysis: pslist, sockstat, bash history, kernel module checks, and TTY activity. Profile detection uses linux.info with linux.kallsyms as fallback.

STEGANOGRAPHY DETECTION

Stegseek performs dictionary attacks against steghide-protected files at extreme speed — testing millions of passwords per second. It can crack weak steganographic passwords in seconds, revealing hidden messages, embedded files, or exfiltrated data concealed within innocent-looking image files.

AUDIT & CHAIN OF CUSTODY

Every tool execution is timed with start/end timestamps. Results are saved to structured log files with descriptive naming. The automation mode ZIP export provides a portable, self-contained evidence package suitable for chain of custody documentation, court submissions, and inter-team evidence sharing during incident response.

ENGINEER

FORENSIC ANALYSIS TOOLKIT

1207	10+	13	2
Lines of Code	Tools Integrated	Menu Options	OS Support

Denis Albin

albindeniswork@gmail.com

// END OF DOCUMENT

Built with passion for cybersecurity